

# RReLU#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.activation.RReLU>.

## Description:

Applies the randomized leaky rectified linear unit function, element-wise. Method described in the paper: Empirical Evaluation of Rectified Activations in Convolutional Network. The function is defined as: where  $a$  is randomly sampled from uniform distribution  $U(\text{lower}, \text{upper})$  during training while during evaluation  $a$  is fixed with  $a = \frac{\text{lower} + \text{upper}}{2}$ .  $\text{lower}$  (float) – lower bound of the uniform distribution. Default:  $\frac{1}{8}$

## Function Signature

---

```
class torch.nn.modules.activation.RReLU(lower=0.125,  
upper=0.3333333333333333, inplace=False)[source]#
```

Parameters

Shape:

`extra_repr()`[source]#

Return type

**Returns:**

Tensor

## Code Examples

---

```
>>> m = nn.RReLU(0.1, 0.3)  
>>> input = torch.randn(2)  
>>> output = m(input)
```

# Detailed Documentation

---

## Documentation

Applies the randomized leaky rectified linear unit function, element-wise.

Method described in the paper: Empirical Evaluation of Rectified Activations in Convolutional Network.

The function is defined as:

where  $a$  is randomly sampled from uniform distribution  $U(\text{lower}, \text{upper})$  during training while during evaluation  $a$  is fixed with  $a = \frac{\text{lower} + \text{upper}}{2}$ .

`lower` (float) – lower bound of the uniform distribution. Default:  $\frac{1}{8}$

`upper` (float) – upper bound of the uniform distribution. Default:  $\frac{1}{3}$

`inplace` (bool) – can optionally do the operation in-place. Default: False

Input:  $(*)^{(*)} (*)$ , where  $*$  means any number of dimensions.

Output:  $(*)^{(*)} (*)$ , same shape as the input.

Return the extra representation of the module.

Runs the forward pass.

